DHARMSINH DESAI UNIVERSITY
FACULTY OF TECHNOLOGY

By: 1). Nishith Upadhyay Kiritbhai, CE 140, 19ceuon087
2). Nikunj Hudka Lalitkumar, CE 77, 19ceuos163

Project Title: Online Examination System

Contents:

1). Introduction/Abstarct
2). SRS (Software Requirement Specification
3). Design Documents
4). Implements Details
5). Work Flow/ Layouts
6). Conclusion
7). Limitations
8). Bibliography

Abstract

As we know we're in a global pandemic situation, where any educational institutes cannot take exam offline for students. So a system which can take exams online becomes handy in these times.

Any professor or teacher can upload an exam which can be multiple choice questions and students can attend that exam in a given time. Also it's easy for evaluation and sending them results. Also it reduces the work of professor to check the answers as the system verifies each and every answers given by students following the key provided by the professor.

Using this system, it saves time for the professor and also evaluation for student's marks becomes an easy task.

Introduction

Online Examination System in which there are mainly two users the admin and the

students. The admin can approve the teachers who can add the questions as per their need and marks. The students can enter by signing up and the login just before the exams starts. After the completion of exam they can view their own results easily.

Software Requirements Specifications:

Admin

1.1 Manage Exam
Description: The admin manage exam here.

1.1.1 Add Course
Description: The admin can add Course name here.

Input: Course Name, Total no of questions, Total marks.

Output: Course created.

1.1.2 Add Questions
Description: The admin can add questions for their courses.

Input: Question statement, Marks, Four Options, Answer.

Output: Questions saved.

1.1.3 View Marks
Description: The admin can view marks of the students.

Input: Student name
Output: Marks obtained

1.2 Manage Teacher
Description: Admin manage teachers here

1.2.1 Verify Teacher
Description: Admin can approve teacher here
Input:
Output:

1.2.2 View Teacher
Description: Admin can view approved teacher here.
Input: username

Output: Name, Photo, Mobile No, Address.

1.2.3 Update Teacher
Description: Admin can edit teacher's detail here

Input: First Name, Last Name, Mobile No, Address, Username, Password, Photo.

Output: Save Changes
1.2.4 Remove Teacher
Description: Admin can remove teacher here.

Output: Teacher's username deleted.

1.3 Manage Student
Description: Admin can manage student's activity here.

1.3.1 Update Student data
Description: Admin can update Student's personal data here.

1.3.2 Delete Student
Description: Admin can delete any student here.

Input: Student ID.

Output: Student removed.

<u>Teacher</u>

2.1 Signup
Description: Teacher can signup here by providing details.

Input: Input credentials.

Output: Login Page.

2.1.1 Login
Description: Teacher can login here by providing their valid username and password.

Input: Input credentials.

Output: Teacher’s homepage.

2.2 Manage Exam
Description: Teacher can manage exam here.

2.2.1 Add Exam
Description: Teacher can add exam here.

Input: Exam name, Total Questions, Total Marks.

Output: Exam created.

2.2.2 Add Questions
Description: Teacher can add questions here.

Input: Question statement, Marks, Four Options, Answer.

Output: Questions saved.

Student

3.1 Signup
Description: Student can sign up here by providing details.

3.1.1 Login
Description: Student can login here by providing username and password.

Input: Input credentials.

Output: Student's homepage.

3.2 Appear Exam
Description: Student can appear the exam here provided admin/teacher.

Input: Fill appropriate answers.

Output: Results.

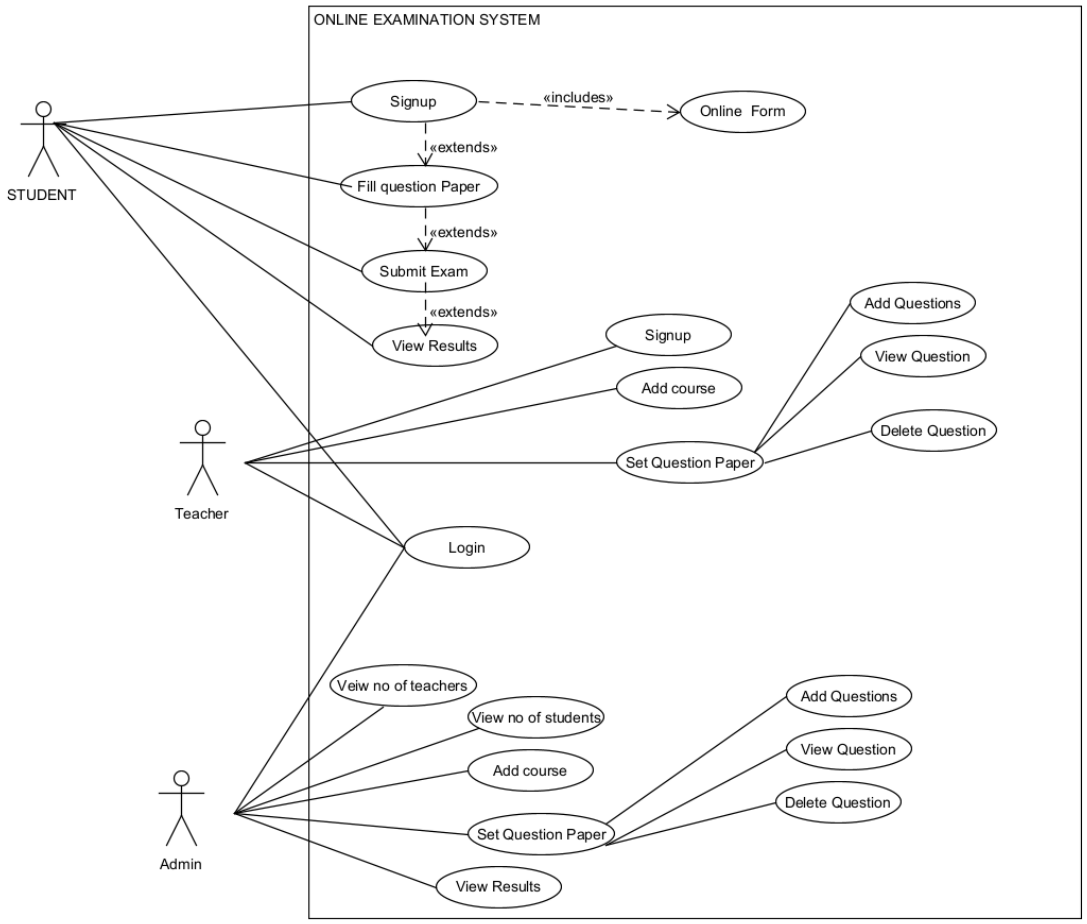
3.2.1 View Marks
Description: Student can view their marks here.

Input:

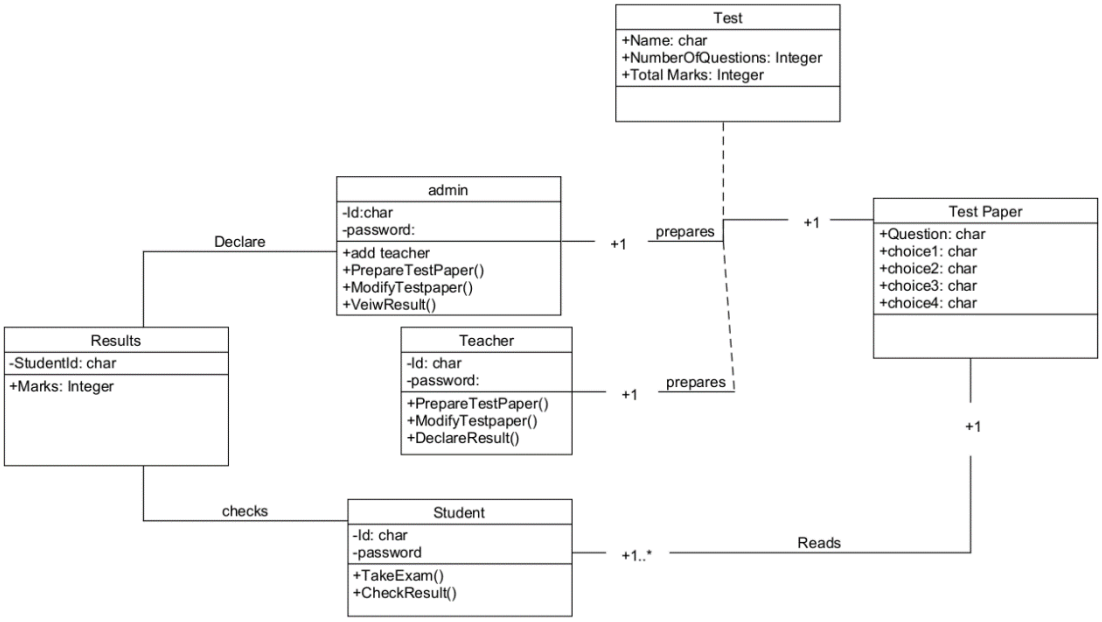
Output: Marks obtained.

Design Documents:

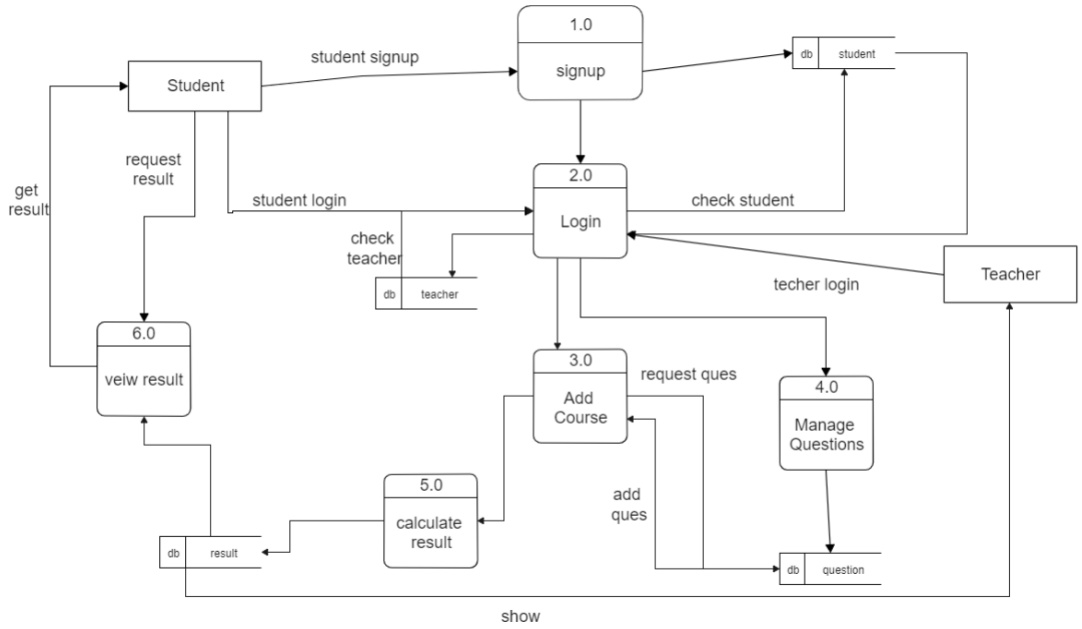
Use Case Diagram:



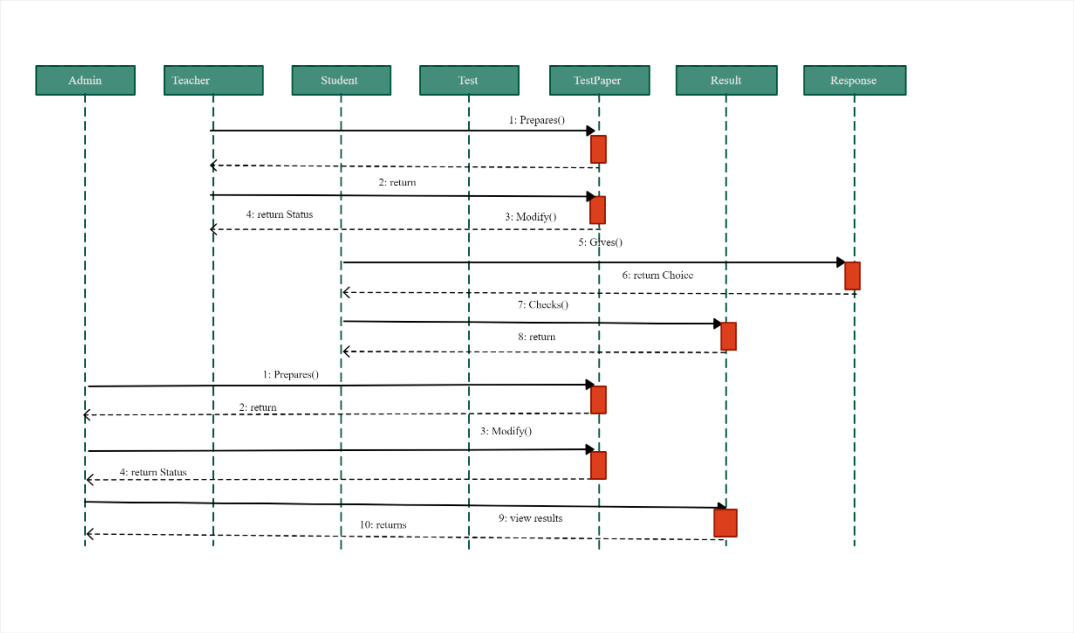
Class Diagram:



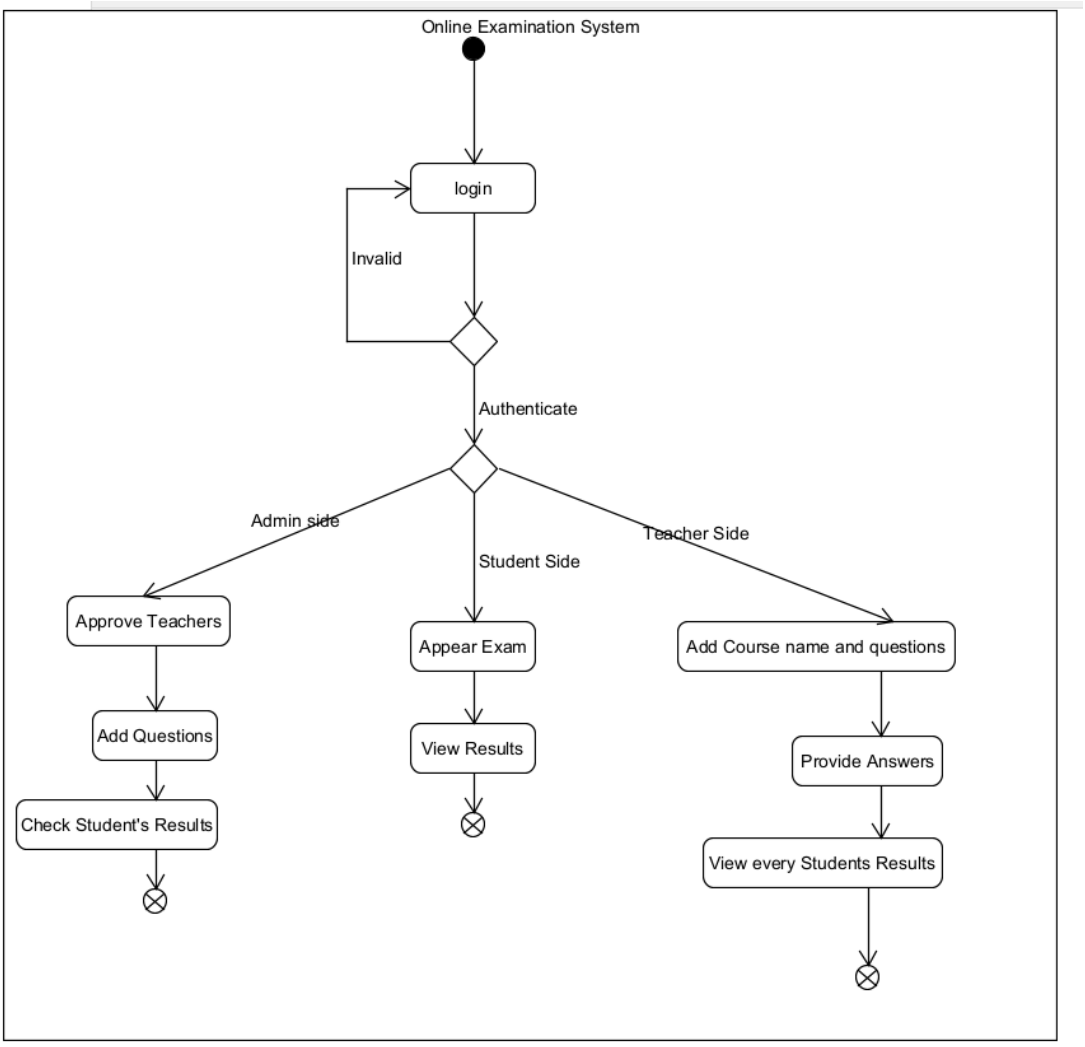
Data flow diagram:



Sequence Diagram:



Activity Diagram:



Implementation Details:

Modules: In this section brief information is given regarding each module.

1). Manage Student-module**:**

In this module, students can appear for the exam provided by the teacher/admin. Also, they can view the results after the completion of the exam.

2). Manage Teacher-module**:**

Here, teacher has to sign up first and then if he/she gets approved by the admin, they can add course name, set of questions, marks, options and the answers.

3). Authentication module:

This module is common for all the users in this system. The admin, teacher, and the student all three has to provide input credentials before entering into the system. Also, they can logout anytime they want to.

4). Manage Exam-module:

This is the main module of our system. The admin/teacher can provide a number of questions as per their wish and each question can carries some marks. Thus, the uploaded question paper can be seen by the students. There is an answer key also provided due to which the responses can be quickly evaluated. So, basically this system works somehow like google forms but has many more functionalities.

Major Functions Prototype:

1). Manage Exam Paper:

This function is used by both admin and the teacher. Here, they can add number of

```python
def admin_question_view(request):
    return render(request,'quiz/admin_question.html')


@login_required(login_url='adminlogin')
def admin_add_question_view(request):
    questionForm=forms.QuestionForm()
    if request.method=='POST':
        questionForm=forms.QuestionForm(request.POST)
        if questionForm.is_valid():
            question=questionForm.save(commit=False)
            course=models.Course.objects.get(id=request.POST.get('courseID'))
            question.course=course
            question.save()
        else:
            print("form is invalid")
        return HttpResponseRedirect('/admin-view-question')
    return render(request,'quiz/admin_add_question.html',{'questionForm':questionForm})


@login_required(login_url='adminlogin')
def admin_view_question_view(request):
    courses= models.Course.objects.all()
    return render(request,'quiz/admin_view_question.html',{'courses':courses})

@login_required(login_url='adminlogin')
def view_question_view(request,pk):
    questions=models.Question.objects.all().filter(course_id=pk)
    return render(request,'quiz/view_question.html',{'questions':questions})

@login_required(login_url='adminlogin')
def delete_question_view(request,pk):
    question=models.Question.objects.get(id=pk)
    question.delete()
    return HttpResponseRedirect('/admin-view-question')
```

questions as their own wish including four options. Also they can remove questions. After

```python
@login_required(login_url='adminlogin')
def admin_view_student_marks_view(request):
    students= SMODEL.Student.objects.all()
    return render(request,'quiz/admin_view_student_marks.html',{'students':students})

@login_required(login_url='adminlogin')
def admin_view_marks_view(request,pk):
    courses = models.Course.objects.all()
    response =  render(request,'quiz/admin_view_marks.html',{'courses':courses})
    response.set_cookie('student_id',str(pk))
    return response

@login_required(login_url='adminlogin')
def admin_check_marks_view(request,pk):
    course = models.Course.objects.get(id=pk)
    student_id = request.COOKIES.get('student_id')
    student= SMODEL.Student.objects.get(id=student_id)

    results= models.Result.objects.all().filter(exam=course).filter(student=student)
    return render(request,'quiz/admin_check_marks.html',{'results':results})
```

the evaluation they can also view the student’s marks.

2). Student's results:

After the completion of the exam students can view their exam's results quickly as the system automatically compares their answers with the answers provided by the admin.

```
def calculate_marks_view(request):
    if request.COOKIES.get('course_id') is not None:
        course_id = request.COOKIES.get('course_id')
        course=QMODEL.Course.objects.get(id=course_id)

        total_marks=0
        questions=QMODEL.Question.objects.all().filter(course=course)
        for i in range(len(questions)):

            selected_ans = request.COOKIES.get(str(i+1))
            actual_answer = questions[i].answer
            if selected_ans == actual_answer:
                total_marks = total_marks + questions[i].marks
        student = models.Student.objects.get(user_id=request.user.id)
        result = QMODEL.Result()
        result.marks=total_marks
        result.exam=course
        result.student=student
        result.save()

        return HttpResponseRedirect('view-result')



@login_required(login_url='studentlogin')
@user_passes_test(is_student)
def view_result_view(request):
    courses=QMODEL.Course.objects.all()
    return render(request,'student/view_result.html',{'courses':courses})
```

```
@login_required(login_url='studentlogin')
@user_passes_test(is_student)
def view_result_view(request):
    courses=QMODEL.Course.objects.all()
    return render(request,'student/view_result.html',{'courses':courses})


@login_required(login_url='studentlogin')
@user_passes_test(is_student)
def check_marks_view(request,pk):
    course=QMODEL.Course.objects.get(id=pk)
    student = models.Student.objects.get(user_id=request.user.id)
    results= QMODEL.Result.objects.all().filter(exam=course).filter(student=student)
    return render(request,'student/check_marks.html',{'results':results})

@login_required(login_url='studentlogin')
@user_passes_test(is_student)
def student_marks_view(request):
    courses=QMODEL.Course.objects.all()
    return render(request,'student/student_marks.html',{'courses':courses})
```

3). Admin Side Functions:

Admin can approve the teachers who can manage the exams. Also he can do all operations on the question paper provided by the teachers to the students.

```python
@login_required(login_url='adminlogin')
def admin_view_pending_teacher_view(request):
    teachers= TMODEL.Teacher.objects.all().filter(status=False)
    return render(request,'quiz/admin_view_pending_teacher.html',{'teachers':teachers})


@login_required(login_url='adminlogin')
def approve_teacher_view(request,pk):
    teacherSalary=forms.TeacherSalaryForm()
    if request.method=='POST':
        teacherSalary=forms.TeacherSalaryForm(request.POST)
        if teacherSalary.is_valid():
            teacher=TMODEL.Teacher.objects.get(id=pk)
            teacher.salary=teacherSalary.cleaned_data['salary']
            teacher.status=True
            teacher.save()
        else:
            print("form is invalid")
        return HttpResponseRedirect('/admin-view-pending-teacher')
    return render(request,'quiz/salary_form.html',{'teacherSalary':teacherSalary})

@login_required(login_url='adminlogin')
def reject_teacher_view(request,pk):
    teacher=TMODEL.Teacher.ob teacher: Teacher
    user=User.objects.get(id=teacher.user_id)
    user.delete()
    teacher.delete()
    return HttpResponseRedirect('/admin-view-pending-teacher')
```

Conclusion:

Thus, this web app becomes handy when the teacher wants to take any kind of tests for students sitting at their homes and it is easy to manage. Also, it has a simple user interface which is easy to understand for students. Just like google forms admin can add questions

with appropriate options and upload it on the webpage and further the students can access it by signing up. After the completion of the exam they can get their individual results.

Limitations and Future Extensions:

1). Here, a limitation is that we couldn’t apply a timer for the exam. But it can be useful for the teacher to add a timer. So, the students can appear the exam in a given specific period of time.

2). Also, questions for written answers can also be given with some limit of words.